MLOps and LLMOps

Lesson 7 of 12 · Ship AI changes safely, again and again.

In this lesson → ① MLOps levels ② LLMOps: what changes ③ The eval gate ④ Several trials ⑤ Registries and prompts ⑥ Infrastructure as code ⑦ Canary releases

Course project: ShopNest Copilot, the AI support assistant we move to the cloud, step by step.

1 MLOps: three levels



0 = by hand · 1 = continuous training · 2 = CI/CD of the pipeline

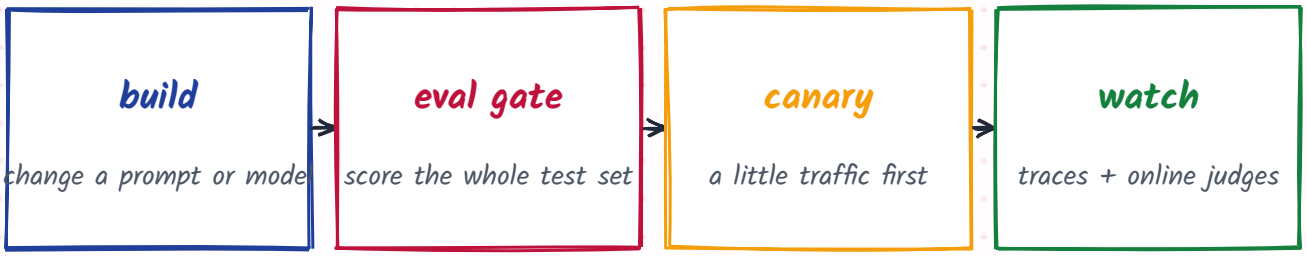
Level	What is automated
0	nothing: notebooks, a model handed over, nobody watching
1	the **training** (continuous training): a pipeline retrains on fresh data, triggered by a schedule, new data or a performance drop
2	**shipping the pipeline**: its code is tested and deployed like any service (CI/CD)

2 LLMOps: what is different

	Classic MLOps	LLMOps
versioned	code, data, features	prompts, model + settings, tools, retrieval index, harness code
quality	one accuracy number	eval suites + live checks
watch	drift	the trace of each request
same input	same output	different outputs: run several trials

CLOUD NOTEBOOK

3 The release loop



every production failure becomes a new test case

Change a prompt or model → **eval gate** scores the whole test set → **canary** on a little traffic → watch traces and online judges → every production failure becomes a new test case.

4 Three layers of tests



each layer catches a different break

→ **code tests**: unit and integration
→ **data + model tests**: training converges, no NaNs, data checked against a reference
→ **LLM eval suites**: graders on a golden set (DeepEval, promptfoo)

5 What a gate checks

- ✓ Challenger beats champion on the holdout by a **margin**, not just matches it
- ✓ No important **slice** gets worse (request type, language, customer group)
- ✓ Speed and size limits are met
- ✓ It loads and serves in a staging container
- ✓ Approval recorded in the registry

6 My toy eval gate

Item	Value
task	route a support ticket to refund / delivery / account / product
"model"	a keyword scorer: a **stand-in**, not a language model
data	100 **synthetic** tickets from templates: 60 delivery, 12 refund, 14 account, 14 product (assumed mix)
v2 change	the word "order" now counts as a delivery signal

Script: `l07_eval_gate.py`. Exit code 1 = blocked, like a failing CI step.

7 The result

delivery	refund	overall
86.7% to 100%	100% to 50%	90% to 92%

the average went UP and the gate still said no

Slice	v1	v2	Change
delivery	86.7%	100.0%	+13.3
refund	100.0%	**50.0%**	**-50.0**
account	100.0%	100.0%	0
product	85.7%	85.7%	0
overall	90.0%	92.0%	+2.0

> GATE: **BLOCKED**: refund dropped 50.0 points. Rules: overall must not fall; no intent may drop by more than 5 points.

Averages hide damage
The word "order" also appears in refund tickets. Always score by slice, and keep a regression suite that must stay near 100%.

8 Capability and regression suites

→ **Regression** evals sit near 100%: any drop means something broke

→ **Capability** evals start low: a hill to climb. When one saturates, it graduates into the regression suite

→ Run evals in CI on every change; monitor after launch; A/B for big changes once traffic allows

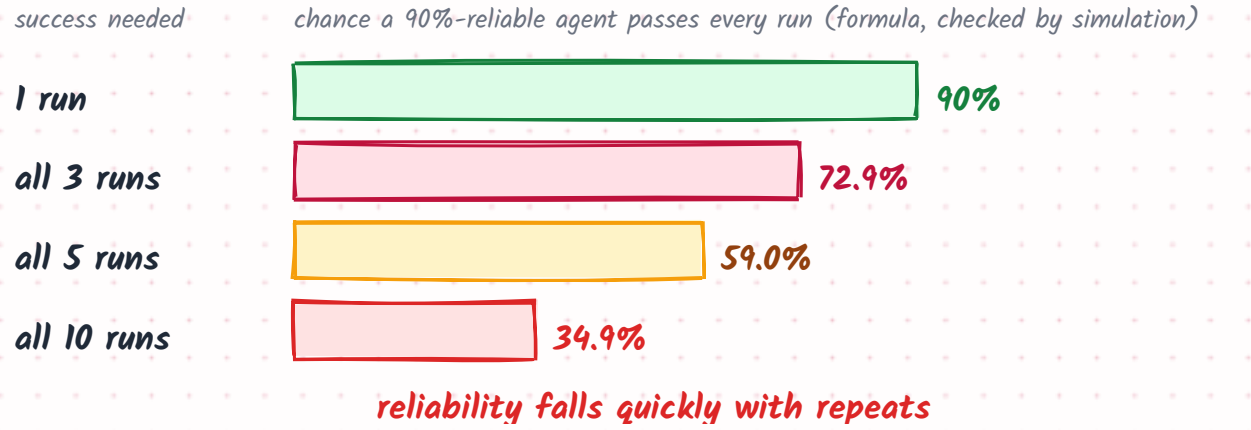
9 Who grades?

- **Error analysis first**: read real failures and group them
- **Code graders** wherever a program can check the answer
- **LLM judge** only for what code cannot check; yes/no questions, calibrated against human labels

10 Why several trials?

> **pass@k = 1 − (1 − p)^k** (at least one of k runs works: can it ever do it?)
> **pass^k = p^k** (all k runs work: can I rely on it?)

11 Reliability, simulated



reliability falls quickly with repeats

p	k	pass@k	pass^k
0.90	1	0.900	0.900
0.90	3	0.999	0.729
0.90	5	1.000	**0.590**
0.90	10	1.000	0.349
0.80	10	1.000	**0.107**

Formulas checked against 100,000 simulated tasks per row (simulated values agree to about 0.003). p is an assumed per-run success rate. Script: L07_trials.py.

12 The gate in CI (sketch)

```
on: pull_request
jobs:
  test-and-eval:
    steps:
      - run: pytest tests/unit          # code
      - run: python checks/data_suite.py  # data
      - run: deepeval test run tests/     # LLM evals
      # any non-zero exit blocks the merge
```

13 Registry and aliases



a rollback is moving the alias, no redeploy

- → A registry answers: which exact version is in prod, who approved it, what did it score? It now tracks models, prompts, judges, eval datasets and tool servers
- → **Aliases, not stages**: point "champion" at one immutable version. Promote or roll back = move the alias
- → The app fetches a prompt by alias and caches it; every trace records the prompt version

14 Model cards

```
---
license: apache-2.0
base_model: BASE_MODEL
base_model_relation: finetune
datasets: [your-org/support-tickets]
tags: [support, lora]
---
# then: intended use, limits, training, results
```

*On Hugging Face the model card is the repo's README. Writing it forces you to say what the model should **not** be used for.*

15 Infrastructure as code



git is the source of truth: rollback = git revert

- → **Terraform / OpenTofu**: network, cluster, GPU pools, permissions, storage
- → **GitOps** (Argo CD, Flux): the cluster is kept equal to what is in Git; rollback = git revert
- → **Rollouts**: canary and blue-green with automatic analysis
- → Keep model-provider keys out of state files (ephemeral and write-only values)

16 A spot GPU pool as code (sketch)

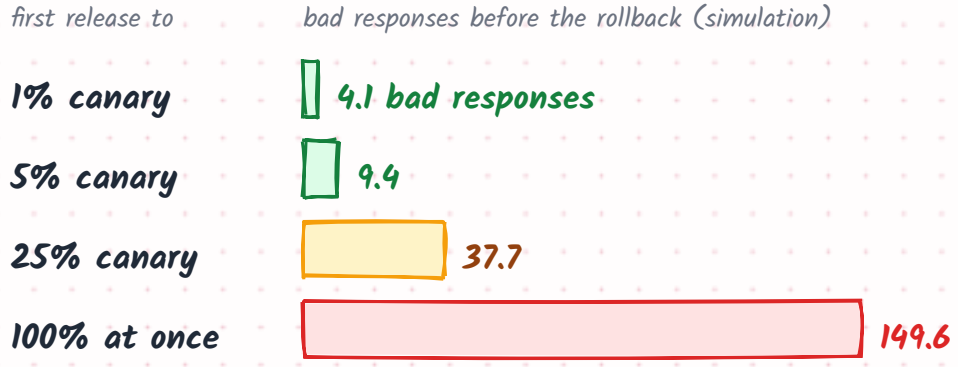
```
resource "google_container_node_pool" "l4_infer" {
  autoscaling {
    min_node_count = 0    # no traffic, no GPU bill
    max_node_count = 4
  }
  node_config { spot = true }  # cheaper, can be taken back
}
```

Illustrative, from the notebook: check the provider docs before use. Ties back to lesson 2 (scale to zero, spot) and lesson 3 (GPU taints).

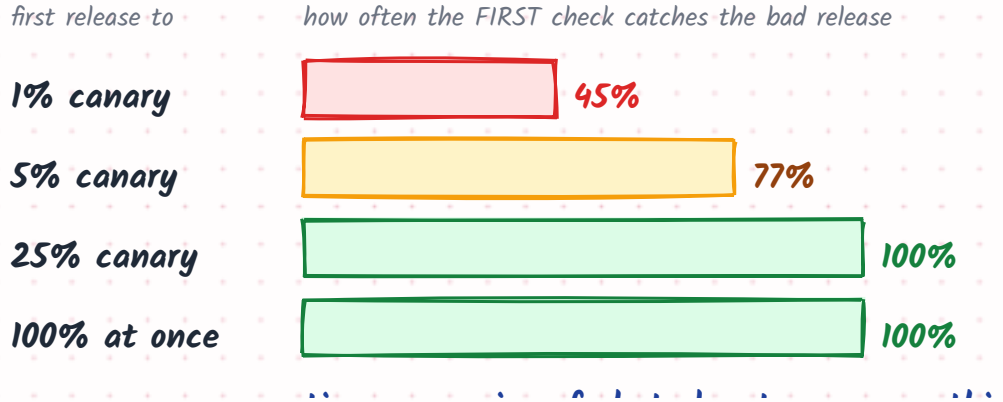
17 Canary: the setup

Setting	Value
traffic	600 requests per minute
old version	fails 1% (assumed)
NEW version	fails **5%** (assumed)
check	every 5 minutes: add up all new-version requests, roll back if z > 3 against 1%
runs	2,000 simulated releases per canary share

18 Canary: the result



Canary share	First check catches it	Mean checks	Bad responses	Requests on new version
1%	45%	2.71	4.1	81
5%	77%	1.25	9.4	188
25%	100%	1.00	37.7	750
100% at once	100%	1.00	**149.6**	3,000



SIMULATION with assumed rates. Script: l07_canary_sim.py. Damage scales with the share you expose. A tiny canary is safe but slow to prove anything: pick a share small enough to be safe and big enough to learn quickly.

19 Observability

- → The unit to watch is the **trace**: one tree per request showing each model call, tool call and retrieval, with time and cost
- → Record traces in the open **OTel GenAI** standard so the backend stays swappable (notebook: Langfuse now owned by ClickHouse, Promptfoo by OpenAI, Helicone in maintenance mode)
- → **Online judges** sample live traces and score them; failures become new test cases
- → **Cost** per request is an alarm next to latency

20 ShopNest Copilot: shipping plan

Piece	Plan
prompts	registry, versions, "production" alias
pull request	unit tests + golden set by slice + cost check
release	5% canary, automatic rollback
watch	traces + online judges, failures become golden cases

21 Words to keep

Word	Meaning
Eval gate	an automatic test that blocks a worse change
Champion / challenger	the current version and the proposed one
Canary	sending a little traffic to the new version first
Trace	the record of everything one request did

22 Interview corner

Q. How do you ship a prompt change safely?

A. Version it, run an eval gate on the overall score and every slice, canary on a small share with automatic rollback, and watch traces. My toy gate blocked a +2 point change that dropped refunds by 50.

Q. Why do averages mislead?

A. A change can help the majority and hurt a minority. Score by slice and keep a regression suite.

Q. Why run evals several times?

A. LLM output varies and reliability falls with repeats: a 90% agent passes 5 runs in a row 59% of the time.

Q. How do you roll back a prompt?

A. Prompts are immutable versions behind an alias; move the alias back. No redeploy.

23 Your turn

107_eval_gate.py: fix version 2 so the gate passes. **107_trials.py**. **107_canary_sim.py**: change the canary share.

⚡ Revise in 60 seconds

- → MLOps levels: 0 by hand · 1 continuous training · 2 CI/CD of the pipeline
- → LLMOps versions prompts, settings, tools, index, harness
- → toy gate: overall **+2.0**, refund **-50**: BLOCKED
- → score by slice; keep a regression suite
- → 90% agent: all 5 runs **59%**, all 10 runs 35%
- → promote or roll back = move an alias
- → canary: 150 bad (100%) vs **9** (5%) vs 4 (1%)
- → watch traces; failures become test cases

Next lesson → CAI-08
GenAI apps: RAG: from documents to cited answers
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**